

**CS2106 Operating Systems
2023/24 Semester II
Midterm Assessment**

1. This paper consists of 20 questions worth 40 marks, printed on ?? (??) printed pages including this page. Please check that you have the correct number of questions and pages.
2. There are two sheets of rough paper attached to the back of this assessment paper. You may use these for your rough work.
3. You are allowed a single cheat sheet printed on both sides, a scientific/business/graphing/regular calculator, 2B pencils and a sharpener.
4. No other electronic devices are allowed.
5. You are provided with an OCR sheet. Please shade AND write your student number on the OCR sheet.
6. OCR sheets without your student number shaded AND written will not be graded and you will receive 0 for this assessment.
7. Do not write your name, contact number or anything else.
8. Shade your answers on the OCR sheet. Ensure that you fully shade your answer.
9. Poorly shaded answers may be incorrectly marked. The teaching staff does not take responsibility for this and will not regrade your script.
10. Do not communicate with each other. If you wish to communicate with an invigilator, raise your hand and keep it raised until you are attended to.

1. Which of the following is true for the layered structure of operating system?
 - A. Lower layer makes use of the operation provided by the upper layers.
 - B. A layered implementation is more efficient than other structures.
 - C. It makes the operating system more difficult to debug and maintain.
 - D. It provides modularity, ease of development, and abstraction of services.
 - E. None of the above options A. to D. are true.

Ans: D
2. Which of the following tasks is typically NOT performed by an operating system?
 - A. Managing system resource such as recycle bin.
 - B. Handling user input such as transferring files.
 - C. Maintaining hardware components such as replacing them.
 - D. Executing application program such as opening files.
 - E. None of the above options A. to D. are correct.

Ans: C
3. Why do CPUs often have different privilege levels (e.g. kernel mode and user mode)? Choose the best answer.
 - A. This allows multiple user processes to run concurrently.
 - B. This allows user-level threads to context switch quickly.
 - C. This restricts user processes from performing potentially harmful or malicious operations that can damage or compromise the system in some way.
 - D. This prevents user processes from forking too many children.
 - E. None of the above options A. to D. are correct.

ANS: C

4. Which of the following statement(s) is/are true about microkernel vs. monolithic operating systems?
 - i. In a monolithic operating system the entire kernel is compiled as a single large program.
 - ii. In a microkernel operating system only the most important parts of the OS run in kernel mode. Everything else runs in user mode.
 - iii. Microkernel OSes provide inter-process communication (IPC) mechanism to allow parts that are operating in user mode to communicate with the kernel, and each other if necessary.
 - iv. The process scheduler in a microkernel OS runs in user mode.
 - v. A poorly written device driver is likelier to crash the entire system in a monolithic OS than in a microkernel OS.
 - A. i., ii, and v. ONLY are true.
 - B. ii. and iii. ONLY are true.
 - C. i., ii. and iv. ONLY are true.
 - D. ii., iii. and v. ONLY are true.
 - E. None of the above options A. to D. are correct.

ANS: A or D

Commented [TKYC1]: Two issues were not clearly communicated during lecture:

i) That while monolithic kernels are “one big program”, they are so in the sense that almost all of the kernel is in kernel mode, it is not necessarily that everything is compiled in one shot. For example in Linux it is possible to plug in new modules into the kernel using insmod, or mounting new filesystems using mkfs, which will not work if indeed everything is done at compile-time. However since this was not clearly communicated I will accept i.

ii) It's also not clearly stated that the IPC is also used for user-process to kernel communications, so iii. may be omitted.

Accepted answers are therefore A. and D.

5. What is a key difference between kernel code (code running in kernel mode) and user code (code running in user mode)?
- A. Kernel code runs in a less privileged mode than user code.
 - B. Kernel code can use a system calls while user code cannot.
 - C. Kernel code does not have direct access to resources while user code does.
 - D. Both user code and kernel code write to the screen only using printf.
 - E. Kernel code runs in a more privileged mode than user code.

Ans: E

6. Which of the following statement(s) is/are true about stack frames?
- i. Storing the return PC on the stack frame allows you to make nested function calls.
 - ii. A stack frame cannot be used if we want to do multithreading.
 - iii. Copying SP to FP lets you use the stack frame for multiprocessing.
 - iv. If you have a pointer x that is a local variable, and it pointing to a memory block allocated using malloc, both x and the memory block are allocated within the heap.
 - v. If you have a local variable y that is declared to be static, then y is allocated in the same memory segment as global variables.
- A. i. and v. ONLY are true.
 - B. i., iv. and v. ONLY are true.
 - C. i., ii. and iii. ONLY are true.
 - D. i., iii. and v. ONLY are true.
 - E. None of the above options A. to D. are correct.

Options i. and v. are true. Answer is A.

7. We are given the following program:

```
int f(int a, int b) {
    int c = 3*a;
    int d = b+2;
    return c + d;
}

...
// Call fn
f(3, 4);
...
```

Assuming that all integers, addresses and registers are 32-bits long, and that the stack-frame convention requires SP and FP to be saved and that dedicated space must be allocated for the return result, what is the minimum size in bytes of the stack-frame for function f, EXCLUDING space needed to temporarily store registers used by f?

Commented [TKVC2]: While the tutorials and notes state that implementations are not fixed, for a practical protocol that permits any number of arguments and supporting nested/recursive function calls, the parameters need to be passed via stack (because of the limited number of registers), and the return address needs to be saved on the stack, which was covered in T1Q3. T1Q4 also states that a proper stack frame implementation will use the stack for parameter passing. Hence 32 remains as the correct answer.

Space needed in bytes:

Return address: 4

FP, SP: 8

a, b: 8

Return value: 4

Local variables c, d: 8

Total: 32 bytes

- A. 20 bytes
- B. 24 bytes
- C. 32 bytes
- D. 40 bytes
- E. None of the above.

Answer: C

8. Suppose a compiler for a MIPS processor uses the following convention on all function calls:

Caller:

Save FP to memory pointed to by SP
Copy SP to FP
Save SP to stack location indicated by FP + 4
Set $SP = SP + 4(n+5)$
Copy parameter 1 to stack location indicated by FP + 12
Copy parameter 2 to stack location indicated by FP + 16
...
Copy parameter n to stack location indicated by FP + 4(n + 3)
jal callee
Copy result from stack location indicated by FP + 4(n + 4)
Copy SP from stack location indicated by FP
Copy FP from stack location indicated by FP + 4

Callee:

Save return address in \$ra to stack location indicated by FP + 8
Set $SP = SP + 4m$
Load parameter 1 from stack location indicated by FP + 12
...
Load parameter n from stack location indicated by FP + 4(n+3)
* Perform Computations using stack as appropriate for local variables*
Store result into stack location indicated by FP + 4(n+4)
Load contents from stack location indicated by FP + 8 into register \$ra
jr \$ra

Which ONE of the following statements is FALSE about this calling convention?

- A. It supports recursive calls.
- B. It supports parameters and global variables of any type.
- C. It saves the return address just after the saved values for SP and FP.
- D. All parameter accesses are done relative to FP.
- E. None of the above statements A. to D. are FALSE (i.e. ALL are true).

Commented [CT3]: It was pointed out that the protocol was buggy and recursion would not work correctly. This is true and so selecting A is acceptable.

Answer: A or B. This convention assumes all parameters and locals are 4 bytes long, which is untrue for char, and too short for double parameters or locals.

9. Consider the following two POSIX threads running concurrently (code to create the threads omitted). No other parts of the program are accessing x in any way. Each operation on x consists of reading x into a register, updating the register, then writing the updated value back to x.

// Global variable int x = 3;	
Thread 1	Thread 2
void *t1(void *p) { x++; }	void *t1(void *p) { x-=2; }

How many possible unique values of x are there? (e.g. if the possible values of x are 3, 4, 4, 5, these count as 3 unique values)

Thread 1: i1:LD R1, X i2:ADD R1, R1, 1 i3:ST R1, X	Thread 2: j1:LD R2, X j2:ADD R2, R2, -2 j3:ST R2, X
--	---

i1i2i3j1j2j3: 2
j1j2j3i1i2i3: 2
i1i2j1j2i3j3: 1
i1i2j1j2j3i3: 4

- A. 2 unique values
- B. 3 unique values
- C. 4 unique values
- D. 5 unique values
- E. None of the above.

Answer: B

10. Consider three processes, all arriving at time zero, with total execution time of 10, 20 and 30 units, respectively. Each process spends the first 10% of execution time doing I/O, the next 70% of time doing computation, 10% of the time doing IO again and the last 10% of time doing computation again. The operating system uses a shortest job first scheduling algorithm (non-pre-emptive) and schedules a new process either when

the running process gets blocked on I/O or when the running process finishes its computation. Assume that all I/O operations can be overlapped as much as possible. For what percentage of time does the CPU remain idle? Choose the closest option.

- A. 2%
- B. 4%
- C. 0%
- D. 6%
- E. 1%

Ans: B

	Total	10% IO	70% Computation	10% IO	10% Computation
P1	10	1	7	1	1
P2	20	2	14	2	2
P3	30	3	21	3	3

I P1 P2 P1 P3 P2 I P3
0 1 8 22 23 44 46 47 50

% idle time = $2/50 * 100 = 4\%$

11. Which of the following statements regarding scheduling algorithms are true?

- A. FCFS discriminates against short jobs since any short jobs arriving after long jobs will have a longer waiting time.
- B. RR treats all jobs equally (giving them equal CPU time units) so short jobs will be able to leave the system faster since they will finish first.
- C. Multilevel feedback queues work similar to the RR algorithm as they discriminate favourably toward short jobs (assume no gaming the system).
- D. SJF selects the process with the smallest next total CPU time, which favours short jobs resulting in a shorter average waiting time.
- E. All the above statements A. to D. are true.

Ans: E

Commented [NL4]: A is true -> If you have a scenario where many long jobs are queued up, and a short job arrive, its waiting time is long. Specifically talking about a scenario where we have many long jobs queued up.

B is true -> in most cases, short jobs leaves system faster.

C is true -> This is example 2 from lecture where short job leaves the system faster similar to RR. It being similar to RR is the case where short job waiting time is bounded, which is true for MLFQ as well.

D is true anyway.

However, given that there are different possible interpretations for the question, we will give +1 mark to everyone.

For Questions 12 and 13, we are given the following CPU bound processes with the arrival and run times shown in this table:

Process	Arrival Time	Run Time
A	0	5
B	0	4
C	3	3

The three processes are running on a variation of Shortest Remaining Time (SRT) and Round Robin, called “Shortest Remaining Burst Round Robin” or SRBRR scheduling.

In SRBRR scheduling, the job with the shortest remaining time is chosen to execute first for a fixed time quantum. After the fixed time quantum, the job with the NEXT SHORTEST remaining time is chosen for execution. If two processes with the same remaining time are both ready to run, the next process is chosen in the order the processes join/rejoin the ready queue.

Here we assume that the time quantum for each process is 2 cycles (or 2 TUs – we take 1 TU = 1 clock cycle).

VER 1

	0	1	2	3	4	5	6	7	8	9	10	11
Run	B4	B3	A5	A4	B2	B1	C3	C2	A3	A2	C1	A1
Ready	A5	A5	B2	B2 C3	C 3 A3	C 3 A 3	A 3	A 3	C 1	C 1	A 1	

VER 2

	0	1	2	3	4	5	6	7	8	9	10	11
Run	B4	B3	A5	A4	C3	C2	B2	B1	A3	A2	C1	A1
Ready	A5	A5	B2	B2 C3	B2 A3	B2 A3	A3 C1	A3 C1	C1	C1	A1	

12. What is the average response time for the three processes?

V1: Average Response Time = $(2 + 0 + (6 - 3)) / 3 = 1.67$ cycles

V2: Average Response Time = $(2 + 0 + (4 - 3)) / 3 = 1$ cycles

- A. 0.65 cycles
- B. 0.83 cycles
- C. 2.67 cycles
- D. 3.25 cycles
- E. None of the above options A. to D. are correct.

Answer: E

13. What is the average turn-around time for the three processes?

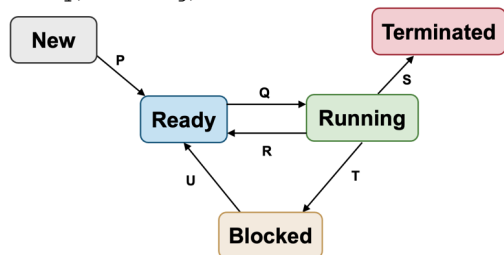
Turn around time = $(12 + 6 + (11 - 3)) / 3 = 8.67$ cycles (Option D)

Turn around time = $(12 + 8 + (11 - 3)) / 3 = 9.33$ cycles (Option E)

- A. 6.33 cycles
- B. 7.67 cycles
- C. 8.33 cycles
- D. 8.67 cycles
- E. None of the above options A. to D. are correct

Answer: D or E

14. In the following process diagram, assume that there is always some process, in Ready, Running, and Blocked state.



Which of the following statements are true?

- A. If a process makes a transition U, it may result in another process making a transition R immediately.
- B. If a process makes a transition S, it may result in another process making a transition P immediately.
- C. If a process makes a transition S, it may result in another process making a transition U immediately.
- D. If a process makes a transition P, it will always result in that process making a transition Q immediately.
- E. None of the above statements are true.

Ans: Two possible answers, both accepted:

A (current running process may get pre-empted by process that made transition U – higher priority/ shorter job),

C (Current process was blocked pending exit of another process using wait system call)

15. Why are threads often called a “lightweight process”? Choose the most appropriate answer.

- A. Threads have their own memory context, separate from the main process.
- B. Threads have their own OS context, separate from the main process.

Commented [CT5]: There was originally a counting error in this question that resulted in choices B or C. The correct choices are D or E.

- C. Threads are executed in parallel, allowing for faster execution compared to processes.
- D. Threads require less overhead to create and manage compared to processes.
- E. None of the above.

Ans. D

16. Consider the following jobs that is scheduled using pre-emptive priority scheduling algorithm (TU = Time Units):

Process	Arrival Time (TU)	Running Time (TU)	Priority (1 highest)
A	0	7	4
B	4	10	1
C	6	3	2
D	8	7	3

What is the waiting time for process A, B, C, and D respectively? *Format in the answer – Process [p(waiting time)]*

[Waiting time includes all time units where the program is ready for execution but could not get the CPU.]

- A. A (20), B (0), C (8), D (9)
- B. A (24), B (4), C (14), D (17)
- C. A (0), B (4), C (14), D (9)
- D. A (20), B (0), C (10), D (13)
- E. A (7), B (0), C (14), D (17)

Ans: A

0 4 14 17 24 27
A B C D A

Waiting time for A => $0 + (24-4) = 20$

Waiting time for B => $(4-4) = 0$

Waiting time for C => $(14-6) = 8$

Waiting time for D => $(17-8) = 9$

17. Consider the following code snippet that executes without any error:

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    for (int i=0;i<3;i++)
        fork();
    printf("CS2106");
}
```

How many times is “CS2106” printed?

- A. 4
- B. 5
- C. 15
- D. 16
- E. 0

Ans: D

$3 + 1 \text{ fork}() = 2^4 = 16$

18. Consider the following code snippet that executes without any error:

```
int main() {
    int i, shmid;
    pid_t pid;
    int *count;
    shmid = shmget(IPC_PRIVATE, sizeof(int), IPC_CREAT |
0600);
    count = (int*) shmat(shmid, NULL, 0);

    count[0] = 0;

    for(i=0; i<=9; i=i+3)
    {
        pid = fork();
        if(pid == 0)
            break;
    }
    if(pid == 0)
    {
        count[0] = i;
        exit(NULL);
    }
    return 0;
}
```

What is final value in count[0]?

- A. 0
- B. 3
- C. 6
- D. 9
- E. Non-deterministic (may give different results for each run)

Ans. E, process can update the value in any order, any of the values are possible.

19. Which of the following statement(s) i. to v. is/ are TRUE about pipes?

- i. If a pipe is full, writing to the pipe results in loss of data.
- ii. A process reading an empty pipe will be blocked until data is available.

- iii. The writer must close the reading end of the pipe and the reader must close the writing end of a pipe to prevent data corruption.
- iv. We can use dup2 to redirect the writing end of the pipe to stdin, and the reading end of the pipe to stdout.
- v. Reading from an empty pipe whose writing end has been closed will result in an end-of-file (EOF) being received.

- A. ii., iii. and iv. ONLY are true.
- B. ii., iv. and v. ONLY are true.
- C. ii. and v. ONLY are true.
- D. i. and iv. ONLY are true.
- E. None of the options A. to D. above are true.

Answer: B (see comment), C

ii and v are true. iv. is incorrect; we would instead replace stdin with the reading end of the pipe and stdout with the writing end of the pipe. This lets us write to the pipe instead of the screen and read from the pipe instead of the keyboard.

We have the following program:

```
#include <pthread.h>
#include <stdio.h>

void *t1(void *p) {
    pthread_exit((void *) 2);
}

void *t2(void *p) {
    pthread_exit((void *) 3);
}

int main() {
    pthread_t a, b;
    pthread_create(&a, NULL, t1, NULL);
    pthread_create(&b, NULL, t2, NULL);

    int c, d;
    void *e, *f;

    c = pthread_join(a, &e);
    d = pthread_join(b, &f);

    printf("Value: %d\n", c + (int) f); // This line
}
```

Assuming that all statements execute without error, what is the value printed at the line labelled "This line"?

Commented [TKYC6]: There is some debate that this should be accepted. The syntax for using dup2 is dup2(oldfd, newfd), which makes newfd point to oldfd. Thus dup2(fd, stdin) redirects a program that would normally read from the keyboard to read from fd, and for this to work with a pipe, we therefore need to dup2 stdin to the READING end of the pipe and not the writing end. Same logic applies to the writing end of the pipe.

Thus linking stdin to the writing end and stdout to the reading end will not work.

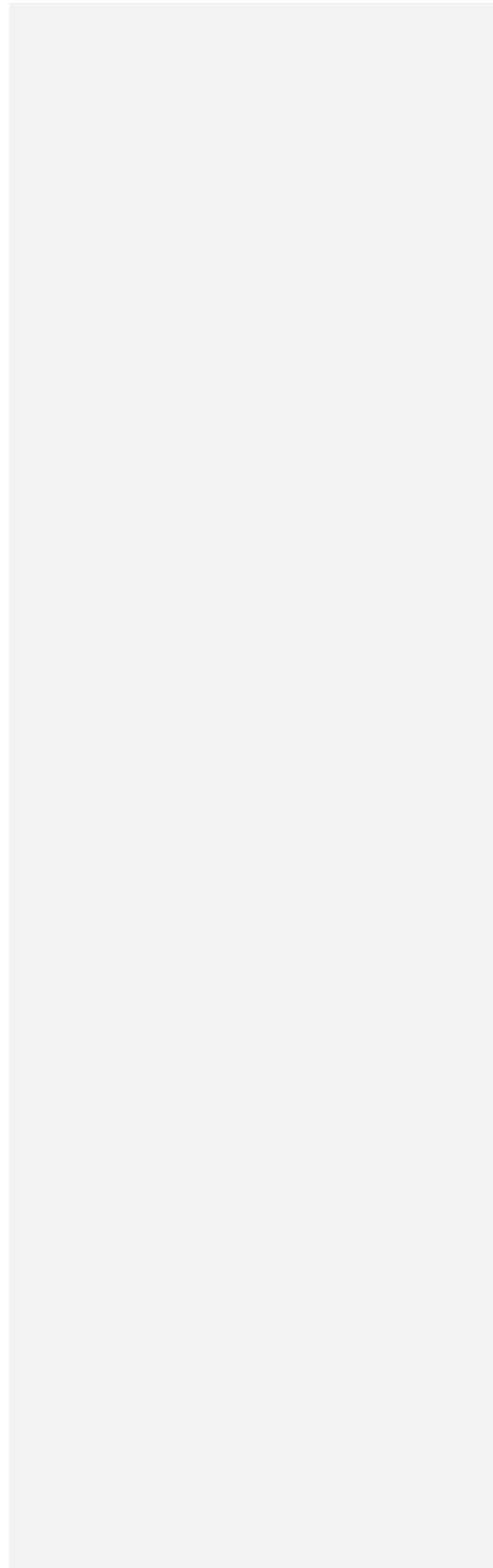
Commented [CT7R6]: However, as I had mistakenly promised to accept B on Piazza, I will honour that promise.

- A. 0
 - B. 2
 - C. 3
 - D. 5
 - E. None of the above options A. to D. are correct.
- Answer: C**

~~ END OF PAPER ~~

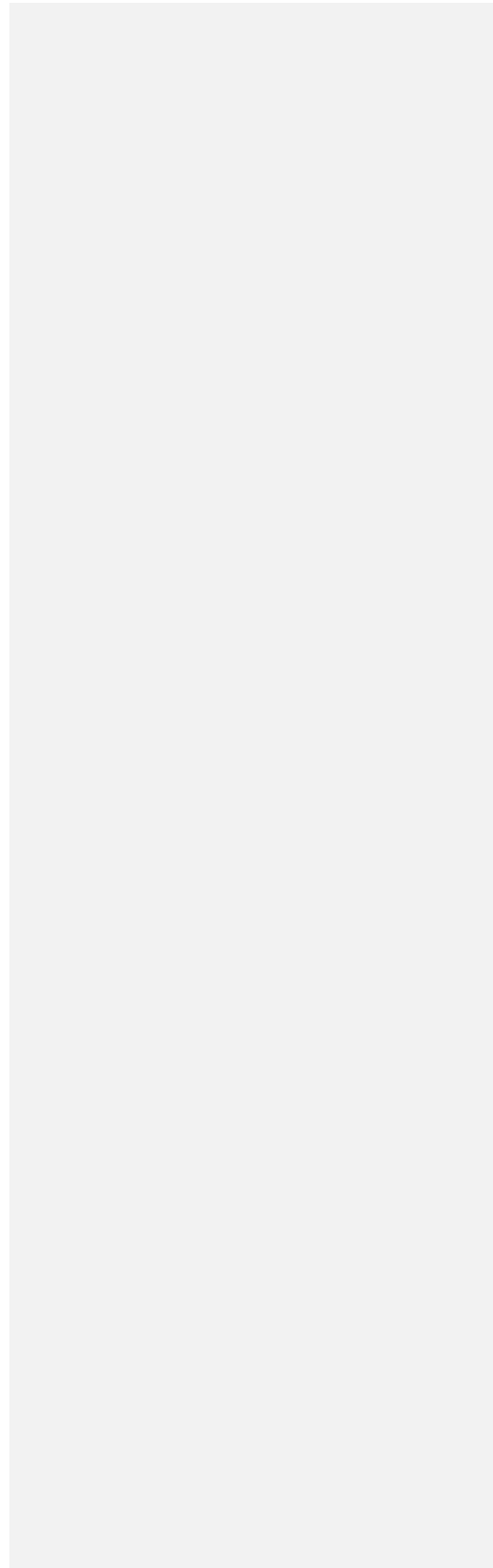
ROUGH PAPER

ROUGH PAPER



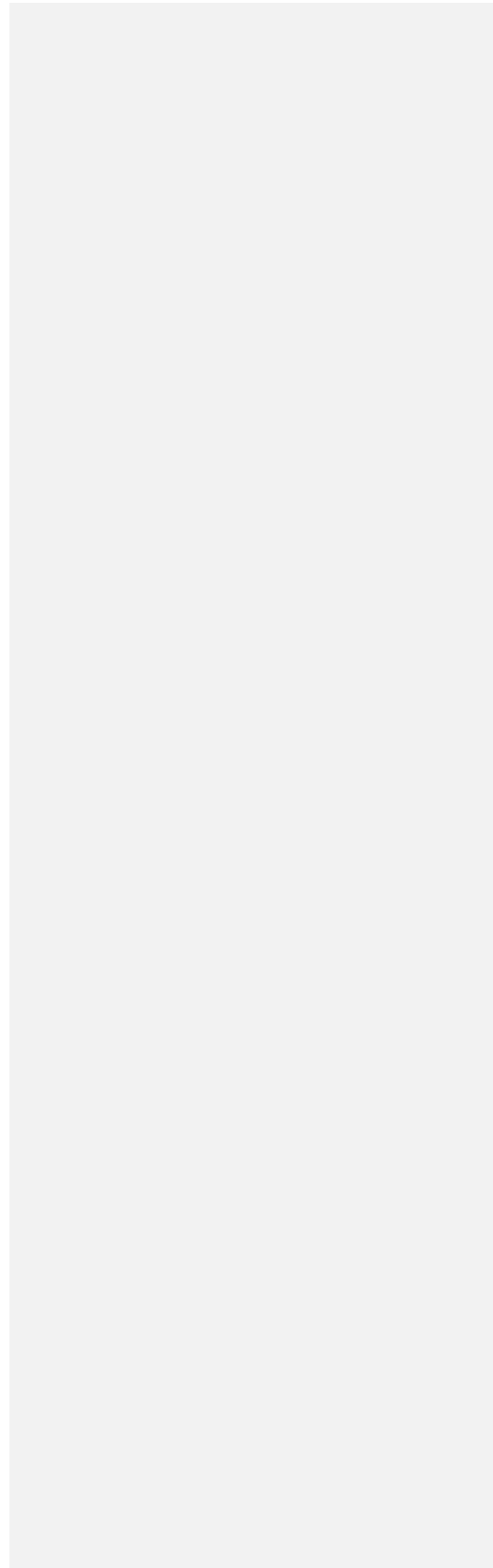
ROUGH PAPER

ROUGH PAPER



ROUGH PAPER

ROUGH PAPER



ROUGH PAPER

ROUGH PAPER

